

Embedded Linux Temperature Monitoring System

Using Raspberry Pi and C

Mohamed Ismail

Contents

1	Introduction	2
2	Objectives	2
3	Hardware Requirements	2
4	Software Requirements	2
5	Hardware Connections	3
6	System Architecture	3
7	Project Workflow	3
8	Linux Concepts Used	4
9	Important Linux System Calls	4
10	Building the Project	4
11	Sample Output	5
12	Program Flow	5
13	Advantages	5
14	Applications	6
15	Future Improvements	6
16	Conclusion	6
17	References	6

1 Introduction

Embedded Linux is widely used in IoT and embedded systems because it provides device drivers, networking, multitasking, and a powerful development environment.

This project demonstrates a Linux-based temperature monitoring system using a Raspberry Pi, an I²C temperature sensor, and the C programming language.

The application communicates with the sensor through the Linux I²C driver, reads temperature values, displays them on the terminal, and stores them in a log file.

2 Objectives

The objectives of this project are:

- Learn Embedded Linux development.
- Interface hardware using Linux.
- Understand Linux device files.
- Access an I²C sensor from a C application.
- Store sensor data in a log file.
- Gain experience with Linux system calls.

3 Hardware Requirements

Component	Description
Raspberry Pi 4	Embedded Linux Board
TMP102 Sensor	Temperature Sensor
MicroSD Card	Raspberry Pi OS Storage
Power Supply	5V USB-C Adapter
Laptop	Development System
Wi-Fi	SSH Connection

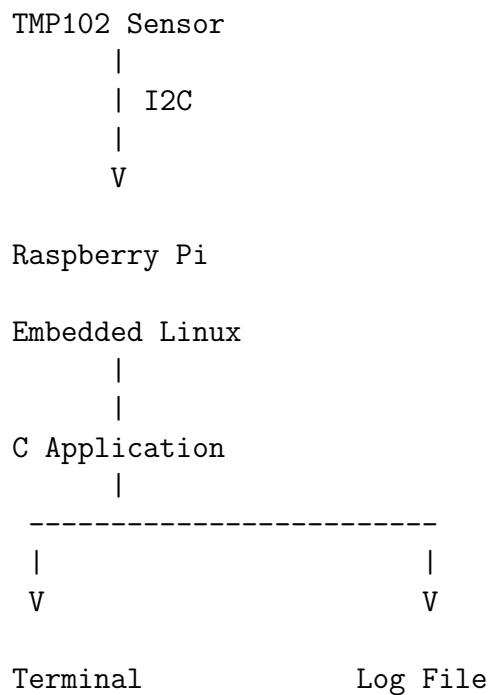
4 Software Requirements

- Raspberry Pi OS
- GCC Compiler
- Linux I²C Driver
- i2c-tools
- SSH
- Git

5 Hardware Connections

TMP102 Pin	Raspberry Pi Pin
VCC	3.3V
GND	GND
SDA	GPIO2
SCL	GPIO3

6 System Architecture



7 Project Workflow

1. Power on Raspberry Pi.
2. Linux boots.
3. Linux loads the I2C driver.
4. Application opens `/dev/i2c-1`.
5. Sensor address is selected.
6. Temperature register is read.
7. Raw data is converted into Celsius.
8. Temperature is displayed.
9. Temperature is stored in a log file.
10. Process repeats every two seconds.

8 Linux Concepts Used

This project demonstrates the following Linux concepts.

- Embedded Linux
- Linux Kernel
- Device Drivers
- Device Files
- I2C Communication
- File Handling
- Process Management
- GCC Compilation
- SSH
- System Calls

9 Important Linux System Calls

The following system calls are used.

Function	Purpose
open()	Open device file
read()	Read sensor data
write()	Select register
ioctl()	Configure I2C device
close()	Release device

10 Building the Project

Compile

```
gcc temperature.c -o temperature_monitor
```

Run

```
./temperature_monitor
```

11 Sample Output

Temperature Monitoring Started

2026-08-01 18:00:01

Temperature : 24.60 C

2026-08-01 18:00:03

Temperature : 24.72 C

12 Program Flow

1. Open Linux I2C device
2. Select sensor
3. Read sensor register
4. Convert raw data
5. Print result
6. Save log
7. Wait
8. Repeat

13 Advantages

- Easy hardware interfacing
- Supports multitasking
- Uses Linux device drivers
- Easy debugging
- Supports networking
- Simple file handling

14 Applications

- Smart Home
- Weather Monitoring
- Industrial Automation
- IoT Devices
- Data Logging
- Environmental Monitoring

15 Future Improvements

Future enhancements include

- MQTT Communication
- Cloud Monitoring
- SQLite Database
- Web Dashboard
- Humidity Monitoring
- Email Alerts

16 Conclusion

This project demonstrates how a Linux user-space application written in C can communicate with an I²C temperature sensor through Linux device drivers. The application reads temperature values, displays them on the terminal, and stores them in a log file. The project provides practical experience with Embedded Linux, Linux system calls, device files, and hardware interfacing.

17 References

1. Raspberry Pi Documentation
2. Linux Kernel Documentation
3. GCC Documentation
4. TMP102 Datasheet